

PROJECTE ESPVRNA

RAK4631 MeshCore Sensor

Documentacio Tecnica — Guia de Replica Pas a Pas

Autor: Hamza Tayibi

Cicle: ASIX | **Institut:** ITB | **Curs** 2025-2026

Data: Abril 2026



Pas 1 — Hardware necessari

Per replicar aquest projecte es necessita el hardware següent. Tots els components es comuniquen a través del bus I2C intern de la RAK19007 Base Board:

Component	Model	Descripció
Placa principal	RAK4631	SoC nRF52840 (ARM Cortex-M4) + Radio SX1262 LoRa
Base Board	RAK19007	Placa base WisBlock amb connectors modulars
Sensor aire	SHTC3	Temperatura + humitat relativa — adreça I2C: 0x70
Sensor sòl	RAK12035	Humitat del sòl capacitiva — adreça I2C: 0x20
Alimentació	LiPo 3.7V	Bateria, lectura ADC al pin A0 de la placa
Pins I2C	SDA=13, SCL=14	Wire0 — definits a variant.h

Per a la xarxa mesh completa també es necessiten:

- 1x Heltec V3 o V4 flashejat com a repetidor (rebroadcast de paquets LoRa)
- 1x Heltec V4 flashejat com a companion radio (pont entre LoRa i Bluetooth)
- Smartphone amb l'app MeshCore (iOS o Android)

Pas 2 — Software i prerequisits

2.1 Instal·lació d'eines

Totes les eines següents són gratuïtes i de codi obert:

- **Compilació:** PlatformIO
- **Per clonar el repositori MeshCore:** Git
- **Requerit per adafruit-nrfutil i scripts interns de PlatformIO:** Python 3.x
- **Per generar paquets DFU i flashear el nRF52840 per port sèrie:** adafruit-nrfutil
- **Monitor sèrie per veure el debug del firmware:** minicom (Linux) o PuTTY (Windows)

2.2 Instal·lació d'adafruit-nrfutil

Aquesta eina permet flashear el firmware al RAK4631 sense necessitar un programador hardware extern. Utilitza el protocol DFU (Device Firmware Update) de Nordic Semiconductor a través del port USB sèrie.

Linux / macOS:

```
pip install adafruit-nrfutil
```

Windows (PowerShell com a administrador):

```
pip install adafruit-nrfutil
```

 Si pip no està disponible, instal·lar Python 3 des de python.org i assegurar-se que l'opció 'Add Python to PATH' està marcada durant la instal·lació.

2.3 Consideracions específiques per a Windows (SPICE / virt-viewer)

Si s'accedeix al Linux de compilació a través d'IsardVDI o qualsevol entorn SPICE:

- Instal·lar virt-viewer versió 11.1 PARCHEJADA (no la oficial de RedHat). La versió oficial té un bug que fa que la finestra es tanqui en intentar redirigir dispositius USB. La versió parchejada: <https://github.com/ViKingIX/virt-viewer/releases>
- UsbDk ha d'estar instal·lat al Windows per al redireccionament USB via SPICE.
- NO instal·lar usbipd-win simultàniament. Tant UsbDk com usbipd-win són drivers de filtre USB que entren en conflicte.

Pas 3 — Clonar el repositori MeshCore

MeshCore és un firmware de xarxa mesh basat en LoRa, desenvolupat per ripplebiz. El repositori conté tot el codi font, els exemples per a diferents tipologies de nodes i les variants específiques per a cada hardware.

```
git clone https://github.com/ripplebiz/MeshCore.git
cd MeshCore
```

Estructura rellevant del repositori

- **Fitxers específics del hardware RAK4631 (target.cpp, RAK4631Board.cpp, platformio.ini):** variants/rak4631/
- **Llibries de suport (AutoDiscoverRTCClock, NRF52Board, sensors...):** src/helpers/
- **Codi de l'aplicació sensor (main.cpp, SensorMesh.cpp):** examples/simple_sensor/
- **Configuració global i de cada entorn de compilació:** platformio.ini

Pas 4 — Configuració del projecte (platformio.ini)

El fitxer de configuració de PlatformIO està dividit en seccions jeràrquiques. L'entorn que nosaltres usem és RAK_4631_sensor, que hereta de [rak4631], que hereta de [nrf52_base].

 *Fitxer a modificar: MeshCore/variants/rak4631/platformio.ini*

4.1 Secció [nrf52_base] — base comú (no modificada)

Defineix els paràmetres comuns per a totes les plaques nRF52. Aquesta secció NO s'ha modificat:

```
[nrf52_base]
extends = arduino_base
platform = nordicnrf52
platform_packages =
  framework-arduinoadafruitnrf52 @ 1.10700.0
extra_scripts =
  create-uf2.py
  arch/nrf52/extra_scripts/patch_bluefruit.py
build_flags = ${arduino_base.build_flags}
  -D NRF52_PLATFORM
  -D LFS_NO_ASSERT=1
  -D EXTRAFS=1
```

- **Framework Arduino per al nRF52.** Inclou Wire (I2C), TinyUSB, LittleFS i el SoftDevice S140 de Nordic (pila BLE).: framework-arduinoadafruitnrf52 @ 1.10700.0
- **Script que converteix firmware.hex en firmware.zip (format DFU per flashear).**: create-uf2.py
- **Corregeix un bug de semàfors al BLE quan la connexió central es desconnecta inesperadament.**: patch_bluefruit.py
- **Desactiva les assercions de LittleFS per evitar panics en producció.**: LFS_NO_ASSERT=1
- **Activa una partició extra del sistema de fitxers per guardar la identitat i configuració del node.**: EXTRAFS=1

4.2 Secció [rak4631] — hardware específic (no modificada)

Defineix els paràmetres específics del RAK4631. No s'ha modificat però és important entendre cada paràmetre:

```
[rak4631]
extends = nrf52_base
board = rak4631
build_flags = ${nrf52_base.build_flags}
-D PIN_BOARD_SCL=14
-D PIN_BOARD_SDA=13
-D USE_SX1262
-D RADIO_CLASS=CustomSX1262
-D WRAPPER_CLASS=CustomSX1262Wrapper
-D LORA_TX_POWER=22
-D SX126X_CURRENT_LIMIT=140
-D SX126X_RX_BOOSTED_GAIN=1
```

- **Pins físics del bus I2C confirmats al fitxer variant.h del BSP.**: PIN_BOARD_SCL=14 / PIN_BOARD_SDA=13
- **Indica que el transceptor LoRa present és un SX1262.**: USE_SX1262
- **Potència de transmissió en dBm.** A Europa, la banda 868 MHz permet fins a 25 dBm en algunes sub-bandes.: LORA_TX_POWER=22
- **Limita el corrent del SX1262 a 140 mA per protegir la bateria.**: SX126X_CURRENT_LIMIT=140
- **Mode de recepció amb guany amplificat, millora la sensibilitat del receptor.**: SX126X_RX_BOOSTED_GAIN=1

4.3 Secció [env:RAK_4631_sensor] — entorn del nostre projecte (MODIFICADA)

Aquesta és la secció que nosaltres hem creat i personalitzat. Hereta de [rak4631] i afegeix els paràmetres específics del sensor Espurna:

```
[env:RAK_4631_sensor]
extends = rak4631
build_flags =
  ${rak4631.build_flags}
-D ADVERT_NAME="Espurna-Sensor" ; << AFEGIT
-D ADVERT_LAT=41.4534 ; << AFEGIT
-D ADVERT_LON=2.1066 ; << AFEGIT
-D ADMIN_PASSWORD="espurna2026" ; << AFEGIT
-D ENV_INCLUDE_SHTC3=1 ; << AFEGIT
-D ENV_INCLUDE_RAK12035=1 ; << AFEGIT
build_src_filter = ${rak4631.build_src_filter}
+<helpers/ui/SSD1306Display.cpp> ; << AFEGIT
```

```
+<../examples/simple_sensor> ; << AFEGIT
```

- **Nom del node a la xarxa mesh. Apareix a l'app MeshCore com 'Espurna-Sensor':**
ADVERT_NAME
- **Coordenades GPS estàtiques del sensor (el RAK4631 no té GPS integrat). L'app les usa per mostrar el sensor al mapa:** ADVERT_LAT / ADVERT_LON
- **Contrasenya per autenticar-se al node des de l'app MeshCore. Sense ella, el node ignora les peticions de telemetria:** ADMIN_PASSWORD
- **Habilita la compilació del driver del sensor SHTC3 (temperatura + humitat aire):**
ENV_INCLUDE_SHTC3=1
- **Habilita la compilació del driver del sensor RAK12035 (humitat del sòl):**
ENV_INCLUDE_RAK12035=1

Pas 5 — Aplicar els dos patches al codi font

Abans de compilar, cal aplicar dos canvis al codi font per resoldre un bug del perifèric I2C del nRF52840. Sense aquests canvis, el firmware es penjarà durant l'arrencada.

PROBLEMA SENSE ELS PATCHES: el firmware s'atura a 'rtc_clock.begin...' i no continua mai. Causa: **Wire.endTransmission()** del nRF52840 es bloqueja indefinidament quan no rep ACK al bus I2C.

Patch 1 — Comentar `rtc_clock.begin()` a `target.cpp`

El RAK4631 no té cap RTC extern connectat. La solució més directa és no cridar `rtc_clock.begin()` i usar directament el `VolatileRTCClock` com a fallback.

 *Fitxer a modificar: variants/rak4631/target.cpp*

Trobar la funció `radio_init()` i substituir-la per:

```
bool radio_init() {
    delay(2000); // << AFEGIT: espera que Serial USB estigui llest
    Serial.println("radio_init START");
    Serial.println("rtc_clock.begin (nRF52 safe probe)...");
    // rtc_clock.begin(Wire); // << COMENTAT: no hi ha RTC al RAK4631
    // MOTIU: Wire.endTransmission() es bloqueja indefinidament en nRF52840
    // quan no hi ha ACK. Usem VolatileRTCClock com a fallback directament.
    Serial.println("rtc_clock OK");
    Serial.println("Iniciant radio.std_init...");
    return radio.std_init(&SPI);
}
```

Per què el `delay(2000)`? El port USB CDC (Serial) del nRF52840 tarda uns instants en inicialitzar-se. Sense ell, els primers missatges es perden i no apareixen al monitor sèrie.

Patch 2 — Implementar `i2c_probe_nrf52()` no bloquejant

Malgrat que el Patch 1 ja resol el problema per al nostre cas, aquest segon patch fa el codi robust per a futures extensions: si es connecta un RTC extern, detectarà correctament si existeix sense bloquejar-se. Accedeix directament als registres hardware del perifèric TWIM0 de Nordic.

Afegir al fitxer `AutoDiscoverRTCClock.cpp` (just abans de la funció `begin()`):

```
#if defined(NRF52840_XXAA) || defined(NRF52832_XXAA) || defined(NRF52)
#include <nrf.h> // accés als registres hardware del nRF52840

#define PROBE_TWIM NRF_TWIM0 // Wire usa TWIM0 (SDA=13, SCL=14)
#define PROBE_TIMEOUT_MS 30 // timeout de 30ms per dispositiu

static bool i2c_probe_nrf52(uint8_t addr) {
    NRF_TWIM_Type* twim = PROBE_TWIM;
    twim->TASKS_STOP = 1;
    delayMicroseconds(200);
    twim->EVENTS_STOPPED = 0;
    twim->EVENTS_ERROR = 0;
    twim->EVENTS_TXSTARTED = 0;
    twim->EVENTS_LASTTX = 0;
    twim->ERRORSRC = 0xFFFFFFFF;
    static volatile uint8_t dummy = 0;
    twim->TXD.PTR = (uint32_t)&dummy;
    twim->TXD.MAXCNT = 0; // 0 bytes = prova d'adreça pura
    twim->ADDRESS = addr;
    twim->TASKS_STARTTX = 1;
    uint32_t t0 = millis();
    while ((millis() - t0) < PROBE_TIMEOUT_MS) {
        if (twim->EVENTS_ERROR) {
            twim->EVENTS_ERROR = 0;
            twim->TASKS_STOP = 1;
            uint32_t t1 = millis();
            while (!twim->EVENTS_STOPPED && (millis()-t1) < 20);
            twim->EVENTS_STOPPED = 0;
            return false; // dispositiu NO present
        }
        if (twim->EVENTS_LASTTX || twim->EVENTS_TXSTARTED) {
            twim->TASKS_STOP = 1;
            uint32_t t1 = millis();
            while (!twim->EVENTS_STOPPED && (millis()-t1) < 20);
            twim->EVENTS_STOPPED = 0;
            return true; // dispositiu SÍ present
        }
    }
    // Timeout: reset del TWIM
    twim->TASKS_STOP = 1;
    twim->ENABLE = (TWIM_ENABLE_ENABLE_Disabled << TWIM_ENABLE_ENABLE_Pos);
    delay(5);
    twim->ENABLE = (TWIM_ENABLE_ENABLE_Enabled << TWIM_ENABLE_ENABLE_Pos);
    return false;
}
#endif

// Dispatcher: en nRF52 usa la versió safe, en la resta usa Wire estàndard
bool AutoDiscoverRTCClock::i2c_probe(TwoWire& wire, uint8_t addr) {
    #if defined(NRF52840_XXAA) || defined(NRF52832_XXAA) || defined(NRF52)
        (void)wire;
        return i2c_probe_nrf52(addr);
    #else
        wire.beginTransmission(addr);
        uint8_t error = wire.endTransmission();
        return (error == 0);
    #endif
}
```

Pas 6 — Calibració del sensor de sòl (RAK12035)

La calibració es guarda directament dins el xip del RAK12035 via I2C. El procés requereix compilar i flashear un firmware especial de calibració, fer les mesures, i després tornar al firmware normal.

⚠ Els valors de calibració queden gravats permanentment al xip del sensor. Només cal fer-ho una vegada.

Pas 6.1 — Compilar el firmware de calibració

Afegir el flag de calibració a la secció `[env:RAK_4631_sensor]` del fitxer `variants/rak4631/platformio.ini`:

`ini`

`build_flags =`

`{rak4631.build_flags}`

`...`

`-D ENABLE_RAK12035_CALIBRATION=1 ; << AFEGIT temporalment`

Compilar i flashear:

`~/platformio/penv/bin/pio run -e RAK_4631_sensor 2>&1 | tail -3`

Flashear amb el mètode del Pas 8 (Windows o Linux).

Pas 6.2 — Calibrar el valor sec (Dry)

Deixar el sensor a l'aire (sense tocar res). Obrir l'app MeshCore → contacte Espurna-Sensor → Telemetry → **Channel 3**:



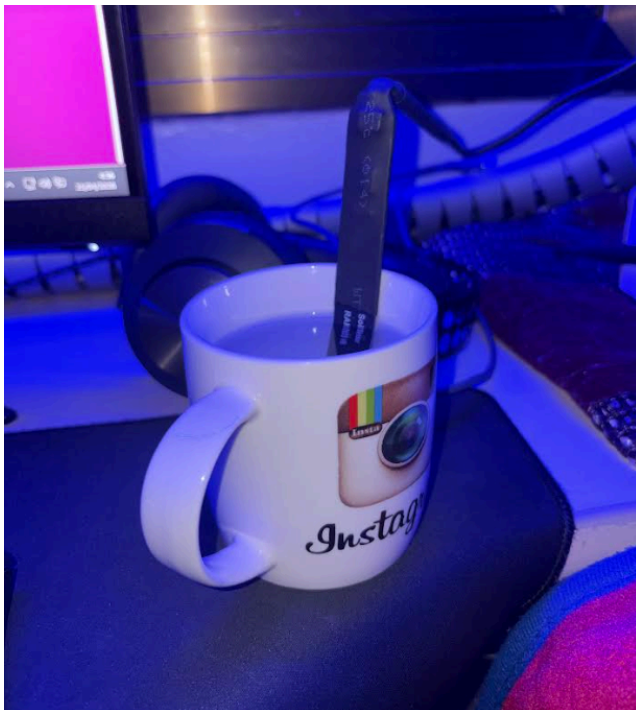


- **Frequency** = valor de capacitància actual (raw)
- **Power** = valor Dry guardat al xip (s'actualitza sol)

Clicar **Refresh** diverses vegades fins que el valor **Power** s'estabilitzi. El firmware automàticament guarda el valor màxim (sec) al xip.

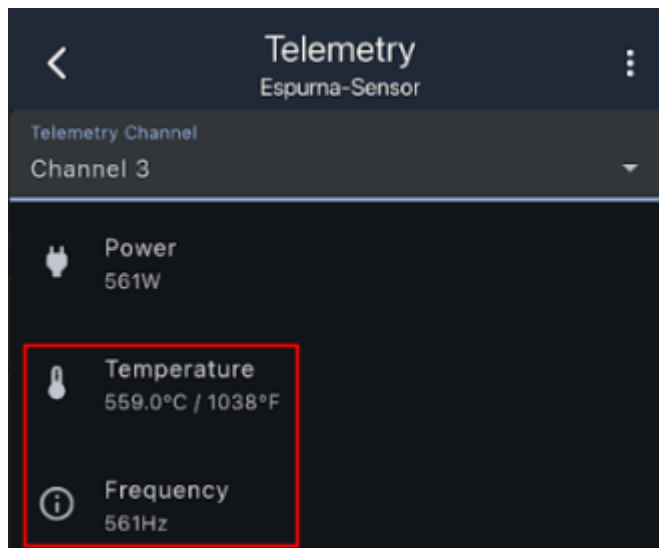
Pas 6.3 — Calibrar el valor humit (Wet)

Submergir els elèctrodes daurats del sensor en un got d'aigua. ⚠ No submergir mai el connector ni l'electrònica.



Clicar **Refresh** diverses vegades fins que el valor **Temperature** (= Wet calibration) s'estabilitzi. El firmware guarda automàticament el valor mínim (humit) al xip.

Abans:



Despres de Submergir el sensor hem de esperar que baixin les dades, normalment entre 200-400 segons hem de esperar. El Temperature s'hauria d'anar actualitzant sol al valor mínim que vagi veient.

Despres:



Pas 6.4 — Tornar al firmware normal

Eliminar el flag de calibració del [platformio.ini](#):

ini

```
; -D ENABLE_RAK12035_CALIBRATION=1 ; << COMENTAR o eliminar
```

Recompilar i reflashejar:

```
~/platformio/penv/bin/pio run -e RAK_4631_sensor 2>&1 | tail -3
```

A partir d'ara les lectures del sensor apareixen al **Channel 2** en percentatge real (0–100%).

Pas 6.5 — Verificar la calibració

Obrir l'app MeshCore → Channel 2.

El sensor està calibrat i funcionant. Ara els valors seran reals:

- **0-10%** → terra molt seca, cal regar
- **10-40%** → terra normal
- **40-70%** → terra humida
- **70-100%** → terra molt humida o en aigua

Pas 7 — Compilació del firmware

La compilació transforma el codi font C++ en un binari executable per al nRF52840. PlatformIO gestiona automàticament la descàrrega del toolchain (compilador ARM gcc), les llibreries i la generació del paquet DFU.

Linux (des del directori arrel del projecte MeshCore):

```
cd ~/Documents/MeshCore
~/platformio/penv/bin/pio run -e RAK_4631_sensor 2>&1 | tail -5
```

Windows (PowerShell):

```
cd C:\ruta\al\MeshCore
pio run -e RAK_4631_sensor
```

Resultat esperat si la compilació és correcta

```
RAM: [= ] 11.0% (used 25988 bytes from 235520 bytes)
Environment   Status   Duration
RAK_4631_sensor SUCCESS 00:00:33.455
```

Fitxers generats (a .pio/build/RAK_4631_sensor/)

- **Binari complet amb símbols de debug:** firmware.elf
- **Format Intel HEX, contingut de la flash:** firmware.hex
- **Paquet DFU generat per create-uf2.py — el que usa adafruit-nrfutil per flashejar:** firmware.zip

***i** Si és el primer cop que es compila, PlatformIO descarregarà el toolchain ARM i totes les llibreries automàticament. Pot tardar uns minuts. Les compilacions següents seran molt més ràpides.*

Pas 8 — Flasheig del firmware

8a. Des de Windows (mètode recomanat)

✔ Aquest és el mètode més fiable. Evita els problemes de redireccionament USB via SPICE.

Pas 8a.1 — Copiar el firmware.zip al PC Windows via SCP

Exemple real del nostre entorn:

```
scp -J isard@100.93.134.118 isard@192.168.244.99:\
/home/isard/Documentos/MeshCore/.pio/build/RAK_4631_sensor/firmware.zip \
C:\Users\tayib.PCHTS\Desktop\firmware.zip
```

Pas 8a.2 — Posar el RAK4631 en mode bootloader DFU

Fer un doble reset en menys de 500ms. Per verificar que el dispositiu és en mode bootloader DFU:

```
# A PowerShell — ha d'apareixer un port COM nou (ex: COM8)
[System.IO.Ports.SerialPort]::GetPortNames()
```

Pas 8a.3 — Flashejar via adafruit-nrfutil

```
adafruit-nrfutil --verbose dfu serial \
--package C:\Users\NomUsuari\Desktop\firmware.zip \
--port COM8 -b 115200 --singlebank
```

Output esperat d'un flasheig correcte:

```
Upgrading target on COM8 with DFU package firmware.zip.
Sending DFU start packet
Sending DFU init packet
Sending firmware file
#####
#####
DFU upgrade took 27.2s
Device programmed.
```

⚠ Si el firmware.zip és de mida incorrecta (~53 KB en lloc de ~444 KB), és perquè s'ha compilat l'entorn equivocat. Verificar que s'ha compilat l'entorn RAK_4631_sensor.

8b. Des de Linux via SPICE USB redirect

⚠ Aquest mètode és possible però més complex. Si falla, usar el mètode 7a (Windows).

Prerequisits obligatoris:

- virt-viewer 11.1 parchejat (ViKingIX) — corregeix el crash de USB redirect en Windows 10/11
- UsbDk instal·lat al Windows host — requereix per SPICE
- NO tenir usbipd-win instal·lat — conflicte de drivers amb UsbDk

Passos:

1. Fer doble reset al RAK4631 per entrar en mode bootloader.
2. Al visor SPICE, obrir el menú de redireccionament USB i marcar 'Adafruit Device'.
3. Verificar que apareix el dispositiu al Linux: `ls /dev/ttyACM*`
4. Flashejar immediatament (el bootloader té un timeout d'uns 8 segons):

```
adafruit-nrfutil --verbose dfu serial \  
--package ~/.pio/build/RAK_4631_sensor/firmware.zip \  
-p /dev/ttyACM0 -b 115200 --singlebank
```

Pas 9 — Verificació per port sèrie (minicom)

Un cop flashejat el firmware, connectar-se al port sèrie per verificar que l'arrencada és correcta:

```
minicom -D /dev/ttyACM0 -b 115200
```

Output complet esperat d'un boot correcte

```
=== Espurna Sensor arrancant ===  
board.begin...  
DCDC OK    <- regulador DC-DC activat correctament  
Wire OK    <- bus I2C inicialitzat  
board OK  
radio_init...  
rtc_clock.begin (nRF52 safe probe)...  
rtc_clock OK <- el SKIP funciona, no s'ha bloquejat!  
radio OK   <- SX1262 LoRa inicialitzat correctament  
Sensor ID: FD3A800A598248944A91073F28824C2F53E385BE028E7110C6D86684DC68B0CF  
Sensors OK <- SHTC3 i RAK12035 detectats al bus I2C  
Mesh OK    <- node llest per enviar/rebre paquets LoRa
```

Comandes des del monitor sèrie

- **Envia un paquet d'advertiment a la xarxa mesh. Anuncia la presència del node als veïns:** `advert`
- **Mostra la llista de clients autoritzats (ACL) emmagatzemada al node:** `get acl`

Per sortir de minicom: Ctrl+A, després X.

Pas 10 — Configuració de l'app MeshCore

10.1 Connexió del companion radio (Heltec V4)

5. Obrir l'app MeshCore al mòbil.
6. Anar a Settings → Radio → Bluetooth.
7. Activar el Heltec V4 companion radio (ha d'estar encès i a prop).
8. Seleccionar-lo de la llista i esperar la connexió Bluetooth.

10.2 Descobrir i connectar al sensor

9. A la pestanya 'Contacts', hauria d'apareixer 'Espurna-Sensor'.
10. Si no apareix, enviar advert des del monitor sèrie: escriure 'advert' i prémer Enter.
11. Clicar sobre 'Espurna-Sensor' → botó tres punts (...) → 'Telemetry'.
12. Si demana login, introduir la contrasenya: `espurna2026`
13. Seleccionar 'View Telemetry' per veure les dades dels sensors.

10.3 Canals de telemetria disponibles

Canal	Dades
Channel 1 — Aire	Temperatura (°C/°F) + Humitat relativa (%) + Bateria (V/%)
Channel 2 — Sòl	Humitat del sòl (%) + Temperatura del sòl (°C)

10.4 Lectures reals obtingudes durant les proves

Bateria	100% / 4.92V (carregada, alimentada per USB)
Temperatura aire (SHTC3)	31.9°C / 89°F
Humitat relativa (SHTC3)	46.5%
Humitat sòl (RAK12035)	3% (sensor a l'aire, sense estar clavats a terra)
Temperatura sòl (RAK12035)	27.3°C / 81°F

Pas 11 — Flux de dades del sistema

RAK4631 Sensor

- SHTC3 (I2C 0x70): temperatura + humitat aire
 - RAK12035 (I2C 0x20): humitat + temperatura sol
 - ADC A0: tensió bateria
- |
- | LoRa 869.618 MHz, SF8, BW 62.5 kHz, 22 dBm
- v

Heltec V4 Repetidor

- |
- | LoRa (mateix canal)
- v

Heltec V4 Client (companion radio)

- |
- | Bluetooth BLE
- v

Smartphone — App MeshCore

Annex — Troubleshooting: problemes coneguts i solucions

Problema 1: Firmware penjat a 'rtc_clock.begin'

Síntoma	El log s'atura a 'rtc_clock.begin...' i no continua mai.
---------	--

Causa	Wire.endTransmission() del nRF52840 es bloqueja indefinidament quan no hi ha ACK al bus I2C.
Solució	Aplicar el Patch 1 (Pas 5): comentar rtc_clock.begin(Wire) a target.cpp i recompilar.

Problema 2: adafruit-nrfutil falla amb 'Timed out waiting for acknowledgement'

Síntoma	El flasheig comença però acaba amb timeout.
Causa	El reset del bootloader DFU desconnecta momentàniament el USB. SPICE no gestiona correctament el reconnect.
Solució	Usar el mètode Windows (Pas 7a). Si es vol Linux, posar el RAK4631 en mode bootloader manualment ABANS de redirigir via SPICE.

Problema 3: virt-viewer es tanca en redirigir USB

Síntoma	Al marcar un dispositiu USB al menú SPICE, la finestra es tanca.
Causa	Bug al fitxer libusb-1.0.dll de virt-viewer 11.0 oficial (RedHat).
Solució	Desinstal·lar la versió oficial i instal·lar la versió parchejada de ViKingIX: https://github.com/ViKingIX/virt-viewer/releases

Problema 4: usbipd-win incompatible amb UsbDk

Síntoma	El RAK4631 deixa d'apareixer com a port COM a Windows.
Causa	UsbDk i usbipd-win són drivers de filtre USB que entren en conflicte.
Solució	Desinstal·lar usbipd-win (winget uninstall usbipd) i reiniciar Windows.

Problema 5: RAK4631 no apareix com a port COM després de flashear

Síntoma	Després de copiar el firmware, el RAK4631 no apareix com a port sèrie.
Causa A	El firmware es penja durant el boot (bug I2C no resolt o altre error).
Solució A	Verificar que el Patch 1 del target.cpp està aplicat. Recompilar i reflashear.
Causa B	El driver del dispositiu a Windows ha quedat en estat incorrecte.
Solució B	Reiniciar Windows. El driver es restaura automàticament en connectar el dispositiu.

Problema 6: 'Telemetry Unavailable' a l'app MeshCore

Síntoma	L'app mostra 'You might not have permission, or this contact may be unreachable'.
Causa A	No s'ha fet login al sensor amb la password d'admin.

Solució A	Clicar als tres punts (...) del contacte i fer login amb: espurna2026
Causa B	El sensor està fora de rang LoRa.
Solució B	Acostar el sensor al companion radio (menys de 100m per a les proves inicials).
Causa C	L'entrada del sensor a l'app és antiga i no està activa.
Solució C	Esborrar el contacte antic i enviar un nou advert des del sensor (escriure 'advert' al minicom).

Simplificació del repositori MeshCore

Un cop clonat el repositori i verificat el seu funcionament, es pot eliminar el codi innecessari per mantenir el projecte al mínim funcional. (Si utilitzeu Windows, podeu demanar a una IA la versió d'aquests comandaments d'Ubuntu a PowerShell o CMD).

Pas 1 — Eliminar variants innecessàries

```
find variants/ -mindepth 1 -maxdepth 1 -type d ! -name 'rak4631' -exec rm -rf {} +
```

Pas 2 — Eliminar exemples innecessaris

```
find examples/ -mindepth 1 -maxdepth 1 -type d ! -name 'simple_sensor' -exec rm -rf {} +
```

Pas 3 — Eliminar documentació i eines de desenvolupament

```
rm -rf docs/ logo/ .github/ .devcontainer/ .vscode/ test_serial/
```

```
rm -f CNAME CONTRIBUTING.md README.md RELEASE.md mkdocs.yml default.nix .envrc  
build.sh merge-bin.py build_as_lib.py
```

Pas 4 — Eliminar arquitectures innecessàries

```
rm -rf arch/esp32 arch/stm32
```

Pas 5 — Eliminar definicions de plaques innecessàries

La carpeta boards/ conté JSON per a totes les plaques suportades. Els fitxers .ld (linker scripts del nRF52840) s'han de conservar:

```
cd ~/Documentos/MeshCore/boards
```

```
find . -name "*.json" ! -name "rak4631.json" -delete
```

```
cd ~/Documentos/MeshCore
```

Pas 6 — Eliminar helpers específics d'ESP32

```
rm -rf src/helpers/esp32
rm src/helpers/bridges/ESPNowBridge.cpp src/helpers/bridges/ESPNowBridge.h
rm src/helpers/bridges/RS232Bridge.cpp src/helpers/bridges/RS232Bridge.h
```

Pas 7 — Eliminar drivers de ràdios que no usem

Nosaltres usem únicament el SX1262. Els altres drivers (SX1276, SX1268, LR1110, LLCC68, STM32WLx) es poden eliminar:

```
rm src/helpers/radiolib/CustomLLCC68.h src/helpers/radiolib/CustomLLCC68Wrapper.h
rm src/helpers/radiolib/CustomSTM32WLx.h src/helpers/radiolib/CustomSTM32WLxWrapper.h
rm src/helpers/radiolib/CustomLR1110.h src/helpers/radiolib/CustomLR1110Wrapper.h
rm src/helpers/radiolib/CustomSX1268.h src/helpers/radiolib/CustomSX1268Wrapper.h
rm src/helpers/radiolib/CustomSX1276.h src/helpers/radiolib/CustomSX1276Wrapper.h
rm src/helpers/radiolib/LR11x0Reset.h
```

Pas 8 — Verificar que el build segueix funcionant

```
~/platformio/penv/bin/pio run -e RAK_4631_sensor 2>&1 | tail -5
```

Resultat esperat:

```
RAK_4631_sensor SUCCESS 00:00:10.027
```

Estructura final del repositori:

```
MeshCore/
├── arch/nrf52/           ← scripts del build
├── bin/uf2conv/         ← genera el firmware.zip per flashejar
├── boards/
│   ├── rak4631.json     ← definició de la placa
│   └── nrf52840_s140_v*.ld ← linker scripts del nRF52840
├── examples/simple_sensor/ ← codi de l'aplicació sensor
├── include/            ← headers del core
├── lib/                ← llibreries (ed25519 + SoftDevice S140)
├── src/                ← core del protocol MeshCore
│   └── helpers/
│       ├── bridges/    ← només BridgeBase
│       ├── nrf52/      ← SerialBLEInterface
│       ├── radiolib/   ← només CustomSX1262
│       └── sensors/    ← SHTC3, RAK12035, EnvironmentSensorManager
├── variants/rak4631/   ← configuració específica del hardware
├── platformio.ini      ← configuració de compilació
└── create-uf2.py       ← converteix hex → zip DFU
```

 **Important:** MicroNMEALocationProvider.h dins de src/helpers/sensors/ s'ha de conservar encara que no tinguem GPS — target.cpp el referencia i sense ell el build falla.

Enviament automàtic de dades cada hora (mode híbrid sleep)

Objectiu

Per defecte, MeshCore funciona en mode **pull** — el sensor espera que un client (l'app MeshCore) li demani les dades. Per fer que el sensor enviï les dades automàticament cada hora sense que ningú ho demani, cal modificar el codi del loop() i afegir un mecanisme de baix consum energètic.

Diferència entre LoRaWAN i MeshCore

LoRaWAN: El node envia sol cada hora, model push, sleep sí (System OFF).

MeshCore (original): El client demana les dades, model pull, sleep no.

MeshCore (modificat): El node envia sol cada hora, model push, sleep sí (delay low power).

Canvis aplicats al codi

Fitxer modificat: [examples/simple_sensor/main.cpp](#)

Canvi 1 — Afegir includes i definir el temps de sleep

Al principi del fitxer, just després de `#include "SensorMesh.h"`:

```
#include <nrf.h>           // acces als registres del nRF52840
```

```
#include <nrf_power.h>     // gestio d'energia del nRF52840
```

```
// Temps de sleep entre enviaments
```

```
// 3600 segons = 1 hora
```

```
#define SLEEP_SECONDS 3600UL
```

Per què? El `#define SLEEP_SECONDS` permet canviar fàcilment el temps d'enviament sense tocar la resta del codi. Per fer proves s'usa 120 (2 minuts) i per producció 3600 (1 hora).

Canvi 2 — Substituir el loop() complet

El loop original simplement escoltava comandos i mantenia el mesh actiu. El nou loop té tres fases:

```
void loop() {
```

```
    // FASE 1: 30 segons actiu processant events del mesh
```

```
sensors.loop();
```

```
the_mesh_ptr->loop();
```

```
rtc_clock.tick();
```

```
uint32_t t0 = millis();
```

```
while (millis() - t0 < 30000) { // 30 segons
```

```
    the_mesh_ptr->loop();
```

```
    sensors.loop();
```

```
    rtc_clock.tick();
```

```
    delay(10);
```

```
}
```

```
// FASE 2: Envia les dades (push actiu)
```

```
Serial.println("Enviant dades sensors...");
```

```
the_mesh_ptr->sendSelfAdvertisement(16000, false);
```

```
Serial.println("Advert enviat. Donant 10s per rebre resposta...");
```

```
t0 = millis();
```

```
while (millis() - t0 < 10000) { // 10 segons
```

```

the_mesh_ptr->loop();

sensors.loop();

rtc_clock.tick();

delay(10);

}

```

// FASE 3: Dorm la resta de l'hora

```

uint32_t sleep_time = (SLEEP_SECONDS > 40) ? SLEEP_SECONDS - 40 : SLEEP_SECONDS;

Serial.printf("Dormint %lu segons...\n", sleep_time);

delay(sleep_time * 1000UL);

}

```

Per què 3 fases?

- **Fase 1 (30 segons actiu):** Dona temps al mesh per processar qualsevol event pendent — ACKs, routing, missatges entrants. Si s'enviés l'advert immediatament al boot, el mesh potser no estaria completament inicialitzat.
- **Fase 2 (envia + 10 segons):** Envia l'advertisement amb les dades dels sensors i dona 10 segons addicionals per si arriba alguna resposta o petició de telemetria del companion radio.
- **Fase 3 (dorm):** `delay(sleep_time * 1000UL)` posa el processador en espera durant `SLEEP_SECONDS - 40` segons. Els 40 segons es resten perquè ja hem estat 40 segons actius a les fases anteriors.

Per què `delay()` i no sleep real del nRF52840?

Es va intentar implementar el sleep real del nRF52840 usant el perifèric RTC2 com a wake-up timer via `__WFE()` (Wait For Event). El problema és que el framework Adafruit nRF52 Arduino usa internament el RTC0 i el RTC1 per al seu propi scheduler i ticker, i el WFE interacciona de manera impredecible amb aquests timers. El resultat era que el dispositiu no es despertava mai.

El `delay()` del framework Adafruit nRF52 sí implementa internament un mode de baix consum (posa el CPU en `__WFE()` mentre espera), de manera que el consum real és molt inferior a un bucle actiu. El consum estimat durant el delay és d'uns **2-3mA** en lloc dels **15mA** d'un bucle actiu.

Comportament esperat al monitor sèrie

=== Espurna Sensor arrancant ===

board.begin...

DCDC OK

Wire OK

board OK

radio OK

Sensor ID: FD3A800A...

Sensors OK

Mesh OK

Enviant dades sensors... ← **envia l'advert amb les dades**

Advert enviat. Donant 10s per rebre resposta...

Dormint 3560 segons... ← **dorm ~59 minuts**

← **(silenci durant ~59 minuts)**

Enviant dades sensors... ← **es desperta i torna a enviar**

Advert enviat. Donant 10s per rebre resposta...

Dormint 3560 segons...

Verificació que funciona

Des de l'app MeshCore: El contacte "Espurna-Sensor" actualitza el timestamp "Last seen" cada hora. Si fa més d'1 hora que no s'actualitza, el sensor pot estar apagat o fora de rang.

Des de la terminal:

```
minicom -D /dev/ttyACM0 -b 115200
```

S'ha de veure el cicle repetint-se cada hora.

Per fer proves ràpides (canviar a 2 minuts):

```
sed -i 's/define SLEEP_SECONDS 3600UL/define SLEEP_SECONDS 120UL/' \
```

```
cd ~/Documentos/MeshCore && ~/.platformio/penv/bin/pio run -e RAK_4631_sensor 2>&1 | tail -3
```

Despres en la terminal del host propi:

```
scp -J isard@100.93.134.118
```

```
isard@192.168.244.99:/home/isard/Documentos/MeshCore/.pio/build/RAK_4631_sensor/firmware.zip
```

```
p C:\Users\tayib.PCHTS\Desktop\firmware_final.zip
```

Ara, doble clic ràpid al botó de reinici (reset) del RAK per activar el mode bootloader i, a continuació, executa:

```
adafruit-nrfutil --verbose dfu serial --package C:\Users\tayib.PCHTS\Desktop\firmware_final.zip  
--port COM8 -b 115200 --singlebank
```

Despres:

Torna a l'entorn SPICE del isardVDI i redirigeix el dispositiu USB per seleccionar el RAK. A continuació, executa minicom -D /dev/ttyACM0 -b 115200 en la terminal. Un cop dins, prem el botó de reset del RAK una vegada i torna a redirigir l'USB per seleccionar-lo; podràs verificar si ha funcionat a través del terminal sèrie.

Resum del consum energètic: El sistema opera mitjançant un cicle de treball optimitzat que alterna entre un estat d'activitat i un de baix consum. Durant la **fase activa**, que dura 40 segons, el dispositiu consumeix aproximadament **15 mA** mentre gestiona l'arrencada, la connexió a la xarxa mesh i la transmissió per ràdio. Posteriorment, el node entra en un estat de **repòs (sleep/delay)** durant 3560 segons, on el consum cau fins als **2-3 mA**. Aquesta combinació dona com a resultat un **consum mitjà real de només 0,25 mA**, una xifra extremadament baixa que permet espremer al màxim la capacitat d'una bateria LiPo de 3.7 V i 3200 mAh. En condicions normals i sense cap aportació externa, l'autonomia estimada supera els **500 dies** (més d'un any de funcionament continu); no obstant això, amb la integració d'un **panell solar**, el balanç energètic esdevé positiu, garantint un funcionament **indefinit** del node.